Databases for data analytics

https://github.com/evidencebp/databases-course/

Normalization

Goal

- We will use a working example in order to understand proper data representation
- We will move a step by step into a better representation
- In each step we will observe the current status, identify a problem, and do an improvement
- The example code

Initial state

```
CREATE TABLE customers (
  id int,
  name varchar(100),
  address varchar(100),
  city_status int,
  orders varchar(100),
  categories varchar(100),
  Pythagorean_theorem varchar(100),
  PRIMARY KEY (id)
);
```

Customers
id
name
address
city_status
orders
categories
Pythagorean_theorem

Step 1

	id	name	address	city_status	orders	categories	Pythagorean_theorem
▶	1	Gidi	Dizengoff 99, Tel-Aviv	9	1,2,3	"VIP"	a^2 + b^2 = c^2
	2	Atzmon Anat	Dizengoff 99/1, Tel Aviv	9	4	"VIP", "New"	a^2 + b^2 = c^2
	3	Meir Suissa	Dizengoff, Tel Aviv-Yaffo	9		"New"	a^2 + b^2 = c^2
	4	Gali Atari	Dizengoff 99, Tel Aviv-Yaffo	9		"new"	a^2 + b^2 = c^2
	5	Pluto	Kibbutz Megiddo	10	5, 6	"VIP"	a^2 + b^2 = c^2

Remove unrelated columns

- The Pythagorean theorem is a humanity beautiful asset but has nothing to do with our customers and therefore better be removed. It violates NF2 “Attributes must depend on the key”

```
CREATE TABLE customers (id int, name varchar(100),
  address varchar(100),
  city_status int,
  orders varchar(100),
  categories varchar(100),
  PRIMARY KEY (id));
```

Step 2

	id	name	address	city_status	orders	categories
▶	1	Gidi	Dizengoff 99, Tel-Aviv	9	1,2,3	"VIP"
	2	Atzmon Anat	Dizengoff 99/1, Tel Aviv	9	4	"VIP", "New"
	3	Meir Suissa	Dizengoff, Tel Aviv-Yaffo	9		"New"
	4	Gali Atari	Dizengoff 99, Tel Aviv Yaffo	9		"new"
	5	Pluto	Kibbutz Megiddo	10	5, 6	"VIP"

Do not use complex attributes (name)

Name is a complex attribute.

That violates the NF1 “Attributes should hold a single atomic value”

Let’s split it into first and last names.

Note that:

1. We should add Gov to Gidi
2. Anat Atzmon’s name was written in a different order than the others.
3. Pluto, the famous dog, do not have a last name, not fitting well in this structure

Use first and last names

```
CREATE TABLE customers (
  id int,
  first_name varchar(100),
  last_name varchar(100),
  address varchar(100),
  city_status int,
  orders varchar(100),
  categories varchar(100),
  PRIMARY KEY (id));
```

Step 3

	id	first_name	last_name	address	city_status	orders	categories
▶	1	Gidi	Gov	Dizengoff 99, Tel-Aviv	9	1,2,3	"VIP"
	2	Anat	Atzmon	Dizengoff 99/1, Tel Aviv	9	4	"VIP", "New"
	3	Meir	Suissa	Dizengoff, Tel Aviv-Yaffo	9		"New"
	4	Gali	Atari	Dizengoff 99, Tel Aviv Yaffo	9		"new"
	5	Pluto		Kibbutz Megiddo	10	5, 6	"VIP"

Do not use complex attributes (address)

Address is a complex attribute.

That violates the NF1 “Attributes should hold a single atomic value”.

Let's split it into street, house number and city.

Note that:

1. We should guess Anat's house number, and lose her apartment number

2. We don't have Meir's house number

3. There are no streets in a Kibbutz

Trun address into city, street, and house

```
CREATE TABLE customers (id int,  first_name varchar(100), last_name varchar(100),
  street varchar(100),
  house_number int,
  city varchar(100),
  city_status int,
  orders varchar(100),
  categories varchar(100),
  PRIMARY KEY (id)
);
```

Step 4

id	first_name	last_name	street	house_number	city	city_status	orders	categories
1	Gidi	Gov	Dizengoff	99	Tel-Aviv	9	1,2,3	"VIP"
2	Anat	Atzmon	Dizengoff	99	Tel Aviv	9	4	"VIP", "New"
3	Meir	Suissa	Dizengoff	NULL	Tel Aviv-Yaffo	9		"New"
4	Gali	Atari	Dizengoff	99	Tel Aviv Yaffo	9		"new"
5	Pluto		NULL	NULL	Kibbutz Megiddo	10	5, 6	"VIP"

Use codes (city)

Note that Tel Aviv is written in many different ways, not representing being the same city.

Yes, that happens way too often in the real world, mainly with unstructured input.

Also, a city status is a function of the city, not of the customer.

We therefore create a table for the cities, with their status.

The customer table will use the city key.

There is **one to many relation** between a city and customers.

Note that the use of code is beneficial **yet not required for normalization.**

Create cities table and refer to it

```
CREATE TABLE cities (
  id int,
  name varchar(100),
  status int, # In cities table, the name status is more suitable than city_status
  PRIMARY KEY (id)
);
```

Refer to cities from customers

```
CREATE TABLE customers (id int,  first_name varchar(100),  last_name varchar(100),
  street varchar(100),
  house_number int,
  city_id int,
  orders varchar(100),
  categories varchar(100),
  PRIMARY KEY (id),
  FOREIGN KEY (city_id) REFERENCES cities(id) # Now people cannot live in non-existing cities
);
```

Step 5

	id	name	status
▶	1	Kibbutz Megiddo	10
	2	Tel-Aviv	9

	id	first_name	last_name	street	house_number	city_id	orders	categories
▶	1	Gidi	Gov	Dizengoff	99	2	1,2,3	"VIP"
	2	Anat	Atzmon	Dizengoff	99	2	4	"VIP", "New"
	3	Meir	Suissa	Dizengoff	NULL	2		"New"
	4	Gali	Atari	Dizengoff	99	2		"new"
	5	Pluto		NULL	NULL	1	5, 6	"VIP"

Use atomic representations (categories)

That violates the NF1 “Attributes should hold a single atomic value”

There is a **many to many relation** between customer and category.

In the current representation

1. it is hard to enforce specific categories (note the 'new' of Gali, not like the other 'New')

2. it is hard to look up categories

It is hard to add attribute to categories

We will add categories table

Add categories table

```
CREATE TABLE categories (
  id int,
  name varchar(100),
  importance int,
  PRIMARY KEY (id),
  constraint Check_importnace CHECK (importance>=1 and importance <= 10)
);
```

Add customers_categories to represent the many to many relations

```
CREATE TABLE customers_categories (
  customer_id int,
  category_id int,
  PRIMARY KEY (customer_id, category_id),
  FOREIGN KEY (customer_id) REFERENCES customers(id),
  FOREIGN KEY (category_id) REFERENCES categories(id)
);
```

Customers no longer refers directly to categories

```
CREATE TABLE customers (id int,  first_name varchar(100),  last_name varchar(100),
  street varchar(100),
  house_number int,
  city_id int,
  orders varchar(100),
  PRIMARY KEY (id),
  FOREIGN KEY (city_id) REFERENCES cities(id) # Now people cannot live in non-existing cities
);
```

Step 6

id	name	importance
1	VIP	10
2	New	8

id	first_name	last_name	street	house_number	city_id	orders
1	Gidi	Gov	Dizengoff	99	2	1,2,3
2	Anat	Atzmon	Dizengoff	99	2	4
3	Meir	Suissa	Dizengoff	NULL	2	
4	Gali	Atari	Dizengoff	99	2	
5	Pluto		NULL	NULL	1	5, 6

customer_id	category_id
1	1
2	1
5	1
2	2
3	2
4	2

Do not use complex attributes (orders)

That violates the NF1 “Attributes should hold a single atomic value”

Customers have a **one to many relation** with orders.

Each customer can have many orders but an order might belong to a single customer.

Let's create a table for them too.

Add orders table

```
CREATE TABLE orders (
  id int, # The order id
  customer_id int,
  purchase_date date DEFAULT (CURRENT_DATE),
  branch varchar(100), # One can add a table for branches
  PRIMARY KEY (id),
  FOREIGN KEY (customer_id) REFERENCES customers(id)
);
```

Note that an order has attribute of its own. In a real schema, the order items should be represented too.

Orders refer to customer using the id

Customers no longer has orders information

```
CREATE TABLE customers (id int,  first_name varchar(100),  last_name varchar(100),
  street varchar(100),
  house_number int,
  city_id int,
  PRIMARY KEY (id),
  FOREIGN KEY (city_id) REFERENCES cities(id) # Now people cannot live in non-existing cities
);
```

Step 7

id	name	importance
1	VIP	10
2	New	8

id	first_name	last_name	street	house_number	city_id	orders
1	Gidi	Gov	Dizengoff	99	2	1,2,3
2	Anat	Atzmon	Dizengoff	99	2	4
3	Meir	Suissa	Dizengoff	NULL	2	
4	Gali	Atari	Dizengoff	99	2	
5	Pluto		NULL	NULL	1	5, 6

customer_id	category_id
1	1
2	1
5	1
2	2
3	2
4	2

id	customer_id	purchase_date	branch
1	1	2026-05-05	Dizengoff, Tel Aviv
2	1	2026-05-05	Dizengoff, Tel Aviv
3	1	2026-05-05	Dizengoff, Tel Aviv
4	2	2026-05-05	Dizengoff, Tel Aviv
5	5	2026-05-05	Kibbutz Megiddo
6	5	2026-05-05	Jerusalem

Impact of our changes

Pros	**Cons**
Easier to retrieve and access data.	We have many tables instead of one
Each value is coded in a unique way.	Codes that should be decoded are less readable
Easier to protect data (e.g., List of code values is enforces.)	Joins are required and they will hurt performance
Format is enforced (e.g., address structure)	Data not in the format is ill represented
	More storage might be needed